



Rethinking Soft Compression in Retrieval-Augmented Generation: A Query-Conditioned Selector Perspective

Yunhao Liu
24210240237@m.fudan.edu.cn
Shanghai Key Laboratory of Data
Science, College of Computer Science
and Artificial Intelligence
Fudan University
Shanghai, China

Zian Jia
24110240035@m.fudan.edu.cn
Shanghai Key Laboratory of Data
Science, College of Computer Science
and Artificial Intelligence
Fudan University
Shanghai, China

Xinyu Gao
23210240161@m.fudan.edu.cn
Shanghai Key Laboratory of Data
Science, College of Computer Science
and Artificial Intelligence
Fudan University
Shanghai, China

Kanjun Xu
24210240355@m.fudan.edu.cn
Shanghai Key Laboratory of Data
Science, College of Computer Science
and Artificial Intelligence
Fudan University
Shanghai, China

Yun Xiong*
yunx@fudan.edu.cn
Shanghai Key Laboratory of Data
Science, College of Computer Science
and Artificial Intelligence
Fudan University
Shanghai, China

Abstract

Retrieval-Augmented Generation (RAG) effectively grounds Large Language Models (LLMs) with external knowledge and is widely applied to Web-related tasks. However, its scalability is hindered by excessive context length and redundant retrievals. Recent research on soft context compression aims to address this by encoding long documents into compact embeddings, yet they often underperform non-compressed RAG due to their reliance on auto-encoder-like full-compression that forces the encoder to compress all document information regardless of relevance to the input query.

In this work, we conduct an analysis on this paradigm and reveal two fundamental limitations: (I) **Infeasibility**: full-compression conflicts with the LLM's downstream generation behavior; and (II) **Non-necessity**: full-compression is unnecessary and dilutes task-relevant information density. Motivated by these insights, we introduce **SeleCom**, a selector-based soft compression framework for RAG that redefines the encoder's role as query-conditioned information selector. The selector is decoder-only and is trained with a massive, diverse and difficulty-graded synthetic QA dataset with curriculum learning.

Extensive experiments show that **SeleCom** significantly outperforms existing soft compression approaches and achieves competitive or superior performance to non-compression baselines, while reducing computation and latency by 33.8%~84.6%.

CCS Concepts

- **Computing methodologies** → **Natural language generation**;
- **Information systems** → *Information retrieval*.

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792700>

Keywords

Retrieval-Augmented Generation, Large Language Models, Context Compression

ACM Reference Format:

Yunhao Liu, Zian Jia, Xinyu Gao, Kanjun Xu, and Yun Xiong. 2026. Rethinking Soft Compression in Retrieval-Augmented Generation: A Query-Conditioned Selector Perspective. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792700>

Resource Availability:

Data, models and code are available at <https://doi.org/10.57967/hf/7616>, <https://doi.org/10.57967/hf/7617>, <https://doi.org/10.5281/zenodo.18349968>.

1 Introduction

Retrieval-Augmented Generation (RAG) enhances Large Language Models (LLMs) by grounding their outputs in external knowledge and is now widely deployed on the Web for tasks such as search, question answering (QA), and interactive assistance [8, 11, 13, 15, 22, 25, 46]. This integration mitigates core limitations of LLMs, such as hallucinations and lack of up-to-date, domain-specific knowledge [3, 23, 39, 41]. Despite its effectiveness, it requires the presence of the full document(s) in the model's context window, substantially inflating the context length. This expansion significantly increases inference cost, drives up latency and amplifies the "lost in the middle" phenomenon [14, 26, 29, 37, 48]. Furthermore, excessively long contexts introduce misleading content, potentially misleading downstream generation [1, 10, 17, 45].

Recently, context compression has emerged as a promising solution to these limitations [16, 27, 40]. Early hard compression methods, which operate at the lexical level, often suffer from limited compression rates [2, 18, 24]. Soft compression instead employs an encoder to map sparse token sequences into dense representations, preserving semantics in a more compact and token-saving form. Despite promising compression ratios, prior work often underperforms non-compression RAG baselines [5, 9, 35]. They tend

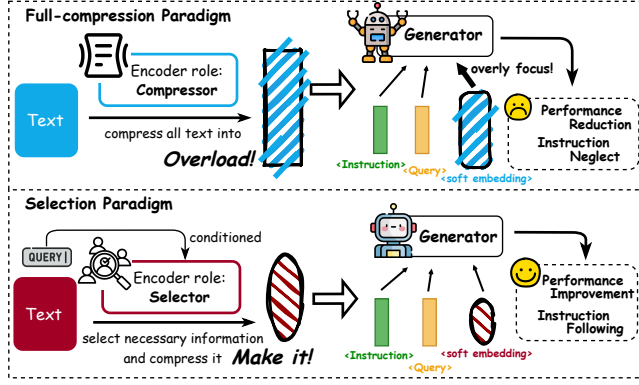


Figure 1: Comparison between Full-compression and Query-conditioned selection. Full document compression causes information losses and overloads the generator towards instruction negligence. The query-conditioned selector extracts only *necessary* information, leading to performance improvement and better query awareness.

to train an encoder-projector complex to compress entire documents into embeddings with an auto-encoder style *full compression* task, which forces the encoder to squeeze *all* document information into a denser embedding. As a result, the compressed document contains redundancy and noise that burden generation, leading to downgraded performance.

Analysis. In this paper, we begin by providing our analysis on this full-compression paradigm. This analysis, supported by theoretical and experimental findings, identifies the root causes of performance limitations in mainstream soft compression:

*Training the encoder towards full information compression is both **infeasible** and **unnecessary**.*

For **infeasibility**, we prove by theory that achieving lossless high-ratio compression into the LLM’s token embedding space is inherently incompatible with its other essential functions, including instruction following and QA. This deduction is also empirically validated: we observe that the full-compression training task causes the compressed embeddings to attract disproportionate LLM attention, blinding the LLM from anything beyond the compressed document. Consequently, the LLM is significantly degraded in its core functionality of query-driven, document-assisted generation.

For **non-necessity**, we observe that under non-compression RAG, the LLM only extracts *necessary* information from the document that is conditioned by the input query (e.g. question), while the majority of the document information receives negligible attention weight. On the other hand, the full-compression encoder tends to allocate considerable attention weight to *all* information without a specific focus. In downstream generation, a harmful bias emerges: the compressed document lacks density in necessary information, but is filled with noisy and unhelpful information. Thus, we argue that given the infeasibility of full compression, to achieve best downstream generation performance, the compressed document should contain query-conditioned essential information only.

Design. Building on the above analysis and observations, we shift the encoder’s role from a full-compressor to a *query-conditioned information selector*, as shown in Figure 1. We propose **SeleCom**, a selector-based soft context compression framework for RAG. It comprises a *selector* that compresses query-conditioned information from the retrieved documents into embeddings, a *projector* that maps them into the generator’s semantic space, and an LLM-based generator that leverages the embeddings to perform RAG.

Different from prior approaches, the selector uses a *decoder-only* backbone. It takes *both query and document* as input, selects and compresses necessary information into embeddings in an *autoregressive* manner. This selection paradigm ensures to *put the most valuable information into limited spaces*, which avoids the infeasibility and non-necessity of full compression simultaneously, achieving a more effective balance between high compression ratios and the generator’s utilization of the retrieved document information.

To fully exploit the potential of **SeleCom**, we first train the selector for effective document context compression. As the selector now performs query-conditioned information selection, it is trained directly towards QA rather than text reconstruction in prior works [5, 9, 42]. Meanwhile, due to lack of document-oriented QA data suitable for this novel task, a comprehensive data synthesis pipeline is adopted to curate a massive, diverse and difficulty-levelled selection training dataset, involving document cleaning, difficulty-aware QA generation and LLM-as-a-judge filtering. Curriculum learning is employed to train the selector with increased difficulty. After the selector is trained, we leverage public QA datasets to teach the generator how to use essential information from embeddings. This two-stage design improves modular decoupling and significantly enhances generation quality.

Evaluation & Contribution. We conducted a comprehensive evaluation of **SeleCom** on six knowledge-intensive tasks, and the state-of-the-art results confirm its effectiveness and efficiency. In terms of performance, **SeleCom** achieves parallel performance with non-compression RAG with significantly reduced latency and inference time. **SeleCom** also surpasses prior state-of-the-art by both effectiveness and efficiency. In addition, large-scale ablation and extended studies further support our design choices. Overall, experimental results consistently demonstrate that **SeleCom** strikes a promising balance among performance and efficiency, highlighting its potential for real-world RAG applications. In summary, our contributions are as follows:

- We provide a principled analysis on soft context compression for RAG, theoretically and empirically demonstrating the **infeasibility** and **non-necessity** of full document compression.
- We shift the role of the context compression encoder into an autoregressive query-conditioned information selector. Motivated by this perspective, we introduce **SeleCom**, a soft context compression framework for RAG. We train **SeleCom** in two-stages, during which we employ a rigorous pipeline to curate a massive and diverse selector training dataset and curriculum learning to gradually increase training difficulty.
- We conduct large-scale experiments across six knowledge-intensive tasks and confirm that **SeleCom** consistently outperforms existing approaches in both performance and efficiency. Ablation studies and diagnostic analyses further support our design choices. We open-source our code, data, and trained models.

2 Related Work

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) has become a standard paradigm for enhancing Large Language Models (LLMs) by grounding their outputs in external, non-parametric knowledge. This approach is particularly effective for knowledge-intensive tasks, as it enables LLMs to incorporate domain-specific, up-to-date, or proprietary information [13, 22]. A typical RAG pipeline begins with a retriever searching a large corpus to identify relevant documents and the generator LLM integrates the original query with the retrieved context to produce a factually-grounded coherent response. Recent research in RAG includes pre- and post-retrieval strategies (e.g. query re-writing [32] and reranking [28]) and more sophisticated RAG pipelines (e.g. Modular RAG [8], Graph RAG [6] and Agentic RAG [3]).

2.2 Context Compression in RAG

Despite its effectiveness, RAG faces practical limitations: incorporating entire documents into the prompt can substantially increase inference costs and may exceed the context length limits of LLMs. To mitigate these issues, context compression techniques have been developed to reduce input length while preserving essential information. These approaches are generally classified into two categories: hard compression and soft compression.

2.2.1 Hard Compression. This approach, also known as lexical-based compression, operates on the surface text, with methods like LLMLingua [18] using smaller models to assess token importance via perplexity, and RECOMP [43] applying a coarse-to-fine strategy to retain key content. While effective in shortening context, such approaches often offer limited compression rate and risk omitting important information.

2.2.2 Soft Compression. In contrast, soft compression has proven more effective. Instead of altering the surface form, it compresses the entire context into a few dense vectors, encompassing various innovative techniques. For instance, ICAE [9] acts as an in-context autoencoder to distill inputs into memory slots; xRAG [5] leverages retrieval embeddings for extreme compression; and COCOM [35] fine-tunes the decoder to interpret learned “context embeddings.” However, these methods largely treat the encoder as a full-context compressor with the aim of encoding all semantics into compact vectors. This overlooks redundancy in retrieved documents and the practical limits of high-ratio compression, which often yields inefficiency and performance reduction. Recently, PISCO [31] utilizes sequence-level knowledge distillation for compression, yet its query-agnostic nature prevents it from filtering out irrelevant information regarding the specific question.

3 Analysis

3.1 The Full Compression Task

We first formalize the task of full compression in the context of RAG. This paradigm typically involves an encoder, f_{enc} , and a generator, f_{gen} . The encoder’s objective is to map an entire long document, $D = \{t_1, \dots, t_L\}$, into vector representations Z :

$$Z = f_{\text{enc}}(D), \quad \text{where } |Z| \ll L$$

To ensure this compact representation preserves the original information, the common reconstruction training task forces the generator to restore the full document D from Z , typically by minimizing the next token prediction loss:

$$\mathcal{L}_{\text{recon}} = - \sum_{i=1}^L \log p(t_i | Z, t_1, \dots, t_{i-1})$$

This reconstruction task implicitly assumes Z to be a “lossless” semantic surrogate for the document, preserving all critical information for any potential query Q . Subsequently, the generator must produce the final answer A relying solely on this highly condensed representation and the query, i.e., $A \sim p(\cdot | Z, Q)$.

3.2 Infeasibility of Full Compression

The infeasibility of full compression does not mean that achieving lossless high-ratio compression is impossible. Instead, it takes the form of an *incompatibility* between full compression and the downstream generation performance of the LLM.

3.2.1 Observation. Given the encoder trained with the full compression task and the frozen LLM used to reconstruct the encoder’s input, we design the following two prompts to test *how the LLM would respond to document-irrelevant instructions*:

Document:

Title: Olaf M. Hustvedt

Content: Hustvedt was married to Irene Cooper Hustvedt (1894–1990), a daughter of the Republic of Hawaii’s co-founder Henry E. Cooper, and was survived by three children. A son, Erling H. Hustvedt (1919–2001), entered the U.S. Naval Academy in 1937 and, although dropped from the academy due to an incorrect evaluation of his eyesight, went on to become a Navy officer, seeing extensive service during World War II. A grandson, Frederick Hauck (b. 1941), became a U.S. Navy officer, fighter pilot, and astronaut in the National Aeronautics and Space Administration’s Space Shuttle Program.

Reconstruction Prompt:

Background: DOCUMENT

Verbatim the background again (including title and content).

Instruction-following Prompt:

Background: DOCUMENT

Ignore the above content and output exactly the following string “SDJKLGHFLKJALPIUOQUY-PUMCUSJKLCOVVILJVHVIFUW”.

As demonstrated in Figure 2(a), we observed a discrepancy in instruction adherence between full-compression and non-compression settings. When given a normal “reconstruct” instruction, the LLM reproduces the document content under both settings. However, when the prompt explicitly asks to ignore the document and output a specific random string (e.g. SDJK...IFUW), the LLM under full-compression setting insists to reconstruct the document, while the LLM under non-compression setting correctly follows the instruction. Diving into the underlying reason behind this observation, we analyze the LLM’s attention weight during its generation. Figure 2(b-e) provides the corresponding attention heatmaps. Under full-compression, the LLM consistently over-focuses on the compressed embeddings, neglecting instruction tokens. In contrast, the LLM under non-compression allocates overwhelming attention weights to the instruction, reflecting faithful instruction adherence.

In other cases, we also find that the LLM, even trained to utilize document embeddings for QA and given document-relevant queries, does not follow query instructions at all (See Appendix I). The fact that the LLM overly focus on document embeddings and neglects any token else explains both this phenomenon, and partially why

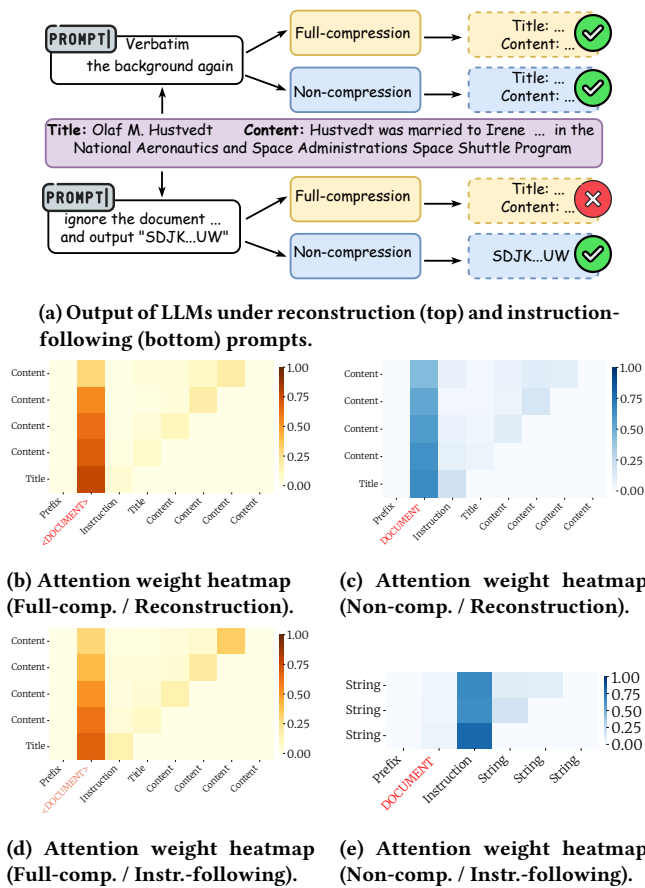


Figure 2: Illustration of instruction (non-)following behaviors under full-compression and non-compression settings.

prior full-compression attempts result in major downstream performance reduction. Next, we proceed to reveal the underlying reason behind this attention score distribution by theory.

3.2.2 Theoretical Explanation. Through theoretical analysis, the underlying mechanism behind this phenomenon can be decomposed into three tightly-linked causes. We explain them in brief, and provide detailed theoretical explanations in Appendix A.1.

(1) The full-compression task first pushes the compressed embeddings Z towards a "positive half-space" of the LLM's token embeddings, causing direction similarity that results in positive attention scores. (2) Once the embedding direction is aligned, the full-compression training pressure polarizes the norm R_Z of the Z to extreme values, amplifying attention scores. (3) As the full-compression task does not enforce instruction-following, attention scores to other tokens remain low. These facts combined drive attention to collapse onto Z across layers and causes the LLM to simply see only the compressed embeddings. The model's ability to analyze the query and extract supporting evidence deteriorates, degrading its downstream performance.

3.3 Non-necessity of Full Compression

We begin by running a controlled comparison between (1) a non-compression RAG pipeline that answers a question given a reference document and (2) an encoder trained with the full compression pipeline that compresses the same reference document into embeddings. The comparison setup is provided below:

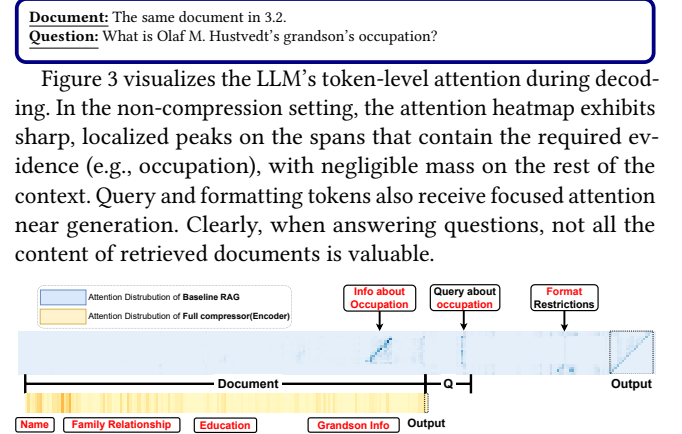


Figure 3: Attention heatmap showing how baseline RAG (blue) and full-compression encoder (yellow) focuses on the input document tokens.

By contrast, the full-compression encoder spreads attention across many irrelevant tokens when generating compressed embeddings. This diffusion dilutes the representation with non-evidence and lowers the concentration of task-relevant cues. Consequently, *the compressed document lacks density in necessary information* and is saturated with noisy, unhelpful signals, which biases downstream generation. This observation support the *non-necessity* of full compression: correct prediction depends on a sparse, query-conditioned subset rather than the entirety of the document.

This non-necessity is also backed by theoretical analysis from the information theory perspective, which we brief and provide the full analysis in Appendix A.2. Let E be the set of necessary information in the document and N be the noisy part. When doing context compression, because the full-compression encoder allocates its attention more averagely across the document, it can be proven that its compressed embedding Z contains less mutual information with E than focusing only on E itself. Meanwhile, due to the presence of noise N , the amount of mutual information further decreases. Thus, if we directly build a selector that directly compresses E into Z , the downstream query-conditioned generation performance will be strictly improved.

4 Methodology

4.1 Preliminaries

The standard RAG pipeline begins with a user-provided query q . A retriever searches a document corpus to find k documents $\{d_{r_1}, \dots, d_{r_k}\}$ most relevant to the query. They are concatenated with the original query and fed into a generator LLM to produce a final answer a . Definitions of the notations we use throughout the paper are provided in Appendix C.

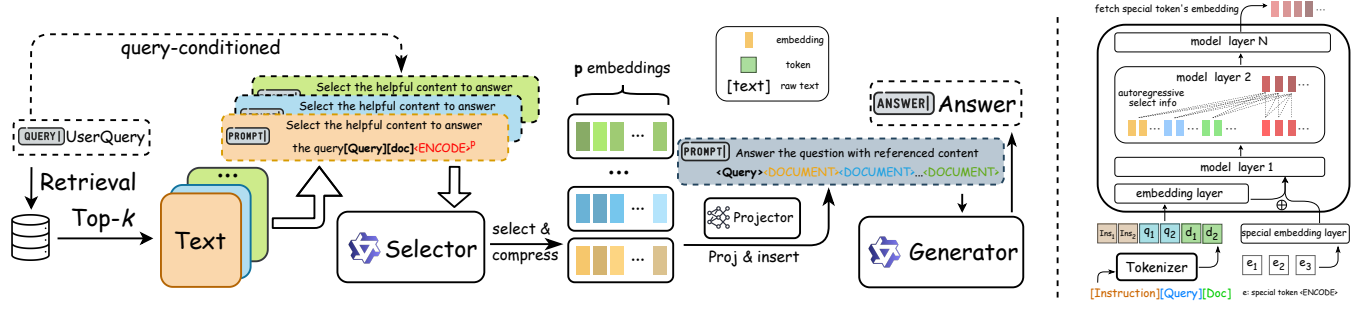


Figure 4: Pipeline overview of SeleCom. Left: the overall workflow—retrieval of top- k documents, query-conditioned selection and compression into p embeddings, projection, and generation of the final answer. Right: the core selector mechanism, where autoregressive generation aggregates information into special-token embeddings with a dedicated, trainable embedding layer.

4.2 Pipeline Overview

As shown in Figure 4, **SeleCom** is a soft context compression framework for RAG that consists of a *selector*, a *projector*, and a *generator*. **SeleCom**’s first step mirrors typical RAG pipelines: corpus construction, index building and document retrieval.

Given a user query and its top- k documents, **SeleCom**’s selector takes both *query* and *document* as its input, extract and compress query-conditioned necessary information from the document into context embedding(s). The selector employs an auto-regressive decoder-only backbone that enforces world knowledge. The projector then aligns the compressed embeddings to the generator’s token embedding space. Taking the query and the compressed embeddings as input, the generator LLM produces the final answer based on the necessary information extracted and embedded in the context embeddings.

SeleCom is trained in a two-stage. The first stage employs direct document-oriented QA with a massive, high-quality synthetic dataset to train the selector on precise and noise-free query-conditioned necessary information selection. The second stage teaches the generator LLM to effectively utilize these information, which is similar to prior works [5, 9].

4.3 Data Construction

To train **SeleCom**’s selector towards query-conditioned necessary information selection, we employ a comprehensive data synthesis pipeline to curate a massive, diverse and difficulty-levelled dataset targetting document-oriented QA. We address the following challenges during data construction: (1) *Document Quality*. Among all documents in the corpus, only those sufficiently long, human-readable and informative ones are suitable as our candidate documents; (2) *QA quality*. While LLMs are the typical solution to QA generation, LLMs often generate questions too easy to answer and wrong answers to the questions.

We choose the Wikipedia dump [15] as our corpus, which contains ~33 million documents. Our overall pipeline begins by filtering overlong documents and discard documents that are too long or too short. Then, we employ LLMs as different roles to consecutively complete document content filtering, document quality filtering, QA generation and QA filtering.

Document content filtering. Many documents in the corpus are actually HTML code of tables, figures and other non-descriptive web page units. We employ LLM-as-a-judge to filter and discard non-human-readable documents, ensuring that the document contains valid information can be used for subsequent QA generation.

Document quality filtering. To make sure the generated questions are meaningful, the documents must contain sufficient information that makes the information extraction process challenging and disturbed by noise. We employ LLM-as-a-scorer to score the information density of the documents into 1-10 and filter documents below the threshold of 6.

QA generation. In this step, we employ LLM-as-a-creator to generate both questions and answers to the documents. The questions are designed to ask for fact information that must be extracted or inferred from the document. We curate two prompt templates to generate QA samples of different difficulty levels, which ensures the diversity of the dataset.

QA filtering. In the final step, we employ LLM-as-a-scorer to: (1) categorize the generated QA difficulty into 1–5 and filter out questions that are either too easy (score 1) or too difficult (score 5); (2) verify and filter wrong answers based on the given document; and (3) divide the remaining QA samples (difficulty 2–4) into three sub-datasets, which are subsequently used for curriculum learning.

After these steps, we curate a massive document-oriented QA dataset containing ~14 million (query, document, answer) pairs, sufficient to train the selector towards query-conditioned information selection. We employ Qwen3-30B-A3B-Instruct-2507 for all LLM-based roles (i.e., judge, creator, and scorer). All prompts used during this process are listed in Appendix J. We also conduct a human verification of this dataset in Appendix H.

4.4 Design detail

4.4.1 Selector. As illustrated in Figure 4, each document d_i is first combined with its corresponding question q_i to form the selection prompt P_{select} (See Appendix J). This prompt is then tokenized into a token sequence T_i augmented with p special <ENCODE> tokens $\langle En \rangle_p = \{En_1, \dots, En_p\}$, forming the input $[T_i; \langle En \rangle_p]$. Embeddings of these special tokens are trainable, avoiding interference with the LLM’s existing embedding space. The sequence is then passed through the selector $Select(\cdot)$ to obtain a sequence of hidden states, whose final p hidden vectors corresponding to the <ENCODE> tokens

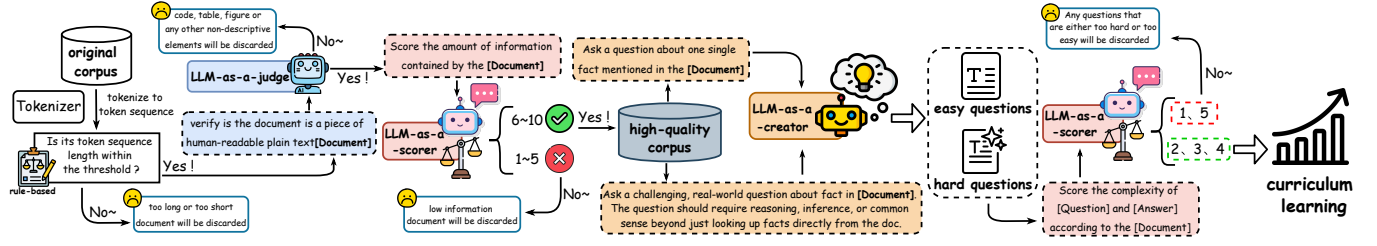


Figure 5: Illustration of the data construction process for the selection training data (Stage1).

are the selector output:

$$H_{1:p}^i = \text{Select}([T_i; \langle En \rangle_p])[\langle En \rangle_p].$$

Unlike traditional bidirectional encoder-only compressors, our selector is generative in nature: the appended `<ENCODE>` tokens integrate global information. This enforces controllable compression by fixing the number of output tokens and aligns more directly with the downstream generation objective. Meanwhile, as $|q_i| \ll |d_i|$, feeding the selector with an extra input query only slightly increases its inference time and computation, which is also empirically validated in §5.3.2.

4.4.2 Projector & Generator. The selector output are concatenated, projected through the projector $\text{Proj}(\cdot)$, and partitioned into n equal-sized latent vectors:

$$E_i = \text{Split}_n(\text{Proj}(\text{Concat}(H_{1:p}^i)))$$

This projector maps compressed embeddings from the selector output space to the generator LLM’s token embedding space, ensuring the generator comprehends the compressed information. The resulting latent embedding set $E_i = \{e_{i,1}, \dots, e_{i,n}\}$ compresses the essential information for answering the query. It takes up to only n tokens in context length, which drastically saves the time and computation during generator decoding. It is then inserted into the generator’s input prompt P_{gen} as n `<DOCUMENT>` tokens for final generation, whose embeddings are exactly replaced by E_i after tokenization and embedding layer:

$$S_i = \text{Emb}(\text{Tok}(P_{\text{gen}}(q_i, \langle \text{DOCUMENT} \rangle * n)))$$

$$\hat{S}_i = \text{Replace}(S_i, \langle \text{DOCUMENT} \rangle, E_i)$$

4.4.3 Stage1: Training Selector and Projector. In this stage, the selector is trained to produce high-quality embeddings that effectively capture the relevant information conditioned by the query, and the projector is trained to align the embedding space of the selector and generator. Only the selector and the projector are updated during this stage, while the generator remains frozen. Given a (q_i, d_i, a_i) pair from the selector training dataset we curate in §4.3, we run (q_i, d_i) through the selector and projector and feeds their output E_i to the generator, allowing it to generate an answer based on the question and reference documents. The objective is to minimize the Next Token Prediction loss between the generator’s output and the correct answer a_i :

$$\mathcal{L} = - \sum_{t=1}^{|a_i|} \log P_{\theta}(a_i^t | S_i, a_1, \dots, a_{t-1})$$

where a_i^t is the t -th token in the answer a_i , and P_{θ} represents the generator’s probability distribution.

We apply curriculum learning on increasing difficulty (§4.3). This stabilizes optimization, allowing the model to progressively develop robust extraction capabilities.

4.4.4 Stage2: Training Information-utilizing Generation. Once the selector and projector have been trained, we focus on training the generator LLM to effectively utilize the inserted compressed embeddings for final generation.

Specifically, we use data from the training set of multiple public QA datasets (q_i, a_i) , obtain their retrieved documents d_i , feed the pair (q_i, d_i) through the selector, projector, and the generator. Similar to Stage 1 training, we calculate the Next Token Prediction loss between the generated answer and the true answer a_i , and use this loss to update the generator. By minimizing this loss, we enable the generator to better utilize the embeddings provided by the selector and projector, thereby generating accurate answers. Prompts used in both training stages can be found at Appendix J.

5 Experiment

5.1 Experimental settings

5.1.1 Datasets. We conduct experiments on: (1) Open-Domain Question Answering (ODQA), including Natural Questions [21], TriviaQA [19], Web Questions [4] and PopQA [33]. (2) Multihop Question Answering, represented by HotpotQA [44]. (3) Fact checking, represented by FactKG [20].

5.1.2 Metrics. Following previous settings [5, 35], we adopt Exact Match (EM) and F1-score (F1). In addition, we employ LLM-as-a-judge (LLM) using Qwen3-30B-A3B-Instruct. This offers greater robustness when answers are *semantically* correct. Detailed prompt in Appendix J. For the FactKG dataset, we use Accuracy (Acc) only [47]. Higher EM, F1, LLM and Acc indicate better performance.

5.1.3 Implementation Details. **SeleCom** applies Qwen3-Embedding-0.6B as the selector backbone, one-layer MLP as the projector, and Mistral-7B-Instruct-0.2/Qwen2.5-7B-Instruct as the generator backbone. We set $p = 8$ and $n = 2$. We define the compression rate as the ratio of the compressed content length to the number of compressed tokens. More details are provided in Appendix B.

5.2 Baselines

We evaluate **SeleCom** against non-compression, representative hard compression baselines and soft compression baselines.

Non-compression Baselines include:

Table 1: Experimental results on six downstream tasks. Top-k indicates the number of input documents. The **best and **second-best** results are highlighted among all top 1 settings. Underlined entries denote the best performance among all top 5 settings. All baselines are tested with their publicly-available model checkpoints under their recommended settings.**

Models	Comp. Rate	Natural Questions			TriviaQA			Web Questions			PopQA			HotpotQA			FactKG
		EM	F1	LLM	EM	F1	LLM	EM	F1	LLM	EM	F1	LLM	EM	F1	LLM	
Mistral-7B-Instruct																	
<i>Top k=1</i>																	
LLM (w/o RAG)	–	4.13	14.30	38.21	5.57	19.18	38.93	4.58	18.14	38.31	9.40	16.06	22.58	5.97	16.28	29.97	58.34
LLM (with RAG)	–	7.98	18.19	40.95	6.41	19.81	40.00	5.36	16.24	31.91	20.91	29.26	38.90	18.69	35.87	58.57	64.16
LLM (with RAG*)	–	<u>41.12</u>	49.86	50.88	35.45	48.72	46.80	<u>30.24</u>	<u>39.40</u>	<u>38.44</u>	<u>30.01</u>	<u>36.27</u>	<u>35.83</u>	<u>42.92</u>	<u>56.43</u>	<u>55.98</u>	61.23
LLMLingua-2	4×	5.87	15.13	35.29	5.72	18.34	37.21	5.12	15.53	27.58	14.02	21.29	29.68	12.46	26.86	45.91	61.65
RECOMP	N/A	8.14	19.06	41.02	7.43	22.13	41.45	N/A	N/A	N/A	N/A	N/A	N/A	18.51	35.76	56.40	N/A
ICAE	4×	14.43	25.99	37.83	9.53	25.03	38.12	9.20	22.36	32.88	26.84	33.80	36.71	24.77	40.06	53.82	62.70
xRAG	164×	3.02	12.45	40.83	5.37	17.92	40.28	2.95	14.22	34.88	10.06	18.30	36.66	8.45	22.52	45.65	63.00
COCOM	128×	31.86	40.15	42.44	13.75	30.68	37.58	20.08	32.19	35.94	21.72	26.21	25.32	25.90	35.36	34.00	60.87
PISCO	64×	0.06	9.49	39.87	0.80	9.11	42.11	0.20	10.24	31.57	0.44	13.28	38.38	0.95	13.08	54.18	65.78
SeleCom	82×	41.80	49.72	50.91	35.26	47.27	45.96	32.04	40.95	40.05	31.42	37.61	37.65	44.46	57.88	57.49	65.23
<i>Top k=5</i>																	
LLM (with RAG*)	–	45.72	54.49	55.73	37.71	50.19	49.06	35.68	44.29	42.28	34.20	40.17	41.05	57.22	74.36	69.96	65.10
SeleCom	82×	44.27	52.85	54.20	<u>37.74</u>	49.83	48.37	33.68	42.04	41.52	33.32	<u>40.51</u>	39.56	<u>61.13</u>	<u>76.16</u>	<u>76.57</u>	<u>66.50</u>
Qwen2.5-7B-Instruct																	
<i>Top k=1</i>																	
LLM (w/o RAG)	–	7.78	17.18	31.51	6.96	19.46	28.60	6.69	20.26	32.74	10.57	15.84	18.16	15.73	23.79	24.63	57.93
LLM (with RAG)	–	15.01	24.96	38.92	9.63	24.13	37.68	9.06	19.29	30.80	25.22	33.19	38.09	36.96	49.95	57.17	62.30
LLM (with RAG*)	–	39.31	48.15	49.91	26.49	41.12	41.07	28.10	37.92	38.16	29.33	35.77	35.13	43.19	52.61	51.71	62.60
SeleCom	82×	38.58	47.02	49.02	30.47	42.43	42.42	27.24	36.94	38.00	31.20	37.71	37.04	42.35	55.34	55.91	67.44
<i>Top k=5</i>																	
LLM (with RAG*)	–	42.76	51.94	54.03	28.95	44.76	46.32	28.69	38.43	39.38	32.28	38.52	37.92	56.26	72.77	74.94	64.60
SeleCom	82×	41.86	50.94	52.22	<u>33.34</u>	<u>45.56</u>	44.69	<u>29.72</u>	<u>39.18</u>	<u>39.93</u>	32.23	<u>38.71</u>	<u>39.02</u>	<u>59.14</u>	<u>74.38</u>	<u>75.43</u>	<u>65.24</u>

Note: RECOMP yields dynamic compression rate due to sentence-level selection. It's dataset-specific checkpoints on Web Questions, PopQA and FactKG are also not available.

- **LLM (w/o RAG)**: relies on internal knowledge to respond.
- **LLM (with RAG)**: standard RAG pipeline using the off-the-shelf frozen LLM for direct inference.
- **LLM (with RAG*)**: standard RAG pipeline fine-tuned on the same dataset as **SeleCom**'s stage2 to ensure a fair comparison, serving as the strongest uncompressed baseline.

Hard compression-based methods include:

- **LLMLingua-2**[34]: Token-level classifier for hard compression.
- **RECOMP**[43]: Sentence-level re-ranking for hard compression.

Soft compression-based methods include:

- **ICAE**[9]: In-context document auto-encoder with a fixed compression ratio.
- **xRAG**[5]: compresses a single document into a single token embedding for subsequent generation.
- **COCOM**[35]: an unified model employs auto-encoding to compress context into embeddings.
- **PISCO**[31]: applies sequence-level knowledge distillation for context compression without annotated data.

5.3 Experimental Results

5.3.1 Performance. Table 1 presents the main results. **SeleCom** consistently outperforms all compression methods and remains competitive with non-compression baselines. Extending retrieval to top-5 further improves performance on multi-hop QA datasets like HotpotQA, showing that multiple compressed embeddings capture complementary evidence without introducing redundancy.

A similar trend holds for Qwen2.5-7B backbone. These results confirm that query-conditioned selection preserves critical reasoning cues even at extreme compression. Additionally, we provide the case study in Appendix I.

5.3.2 Efficiency. We evaluate efficiency with: Total Inference Latency (TIL), measuring end-to-end latency; GFLOPs, approximating the computational cost; and Time to First Token (TTFT), latency to generate the first output token.

As shown in Table 2, **SeleCom** achieves the best efficiency. Compared to non-compression baselines, both GFLOPs and TTFT is significantly lowered due to reduced generator context length. Compared to compression baselines, **SeleCom** also demonstrates superior efficiency due to a lightweight selector. Under top-5, **SeleCom** shows only slightly decreased efficiency, while the baseline's runtime increases sharply, indicating that selective compression scales efficiently with different top-k settings. Efficiency evaluation across all datasets are presented in the Appendix D.

Table 2: Efficiency comparison on the Mistral with respect to LLM (with RAG*). Lower values indicate better efficiency.

Method	TIL (ms)		GFLOPs		TTFT (ms)	
	NQ	HotpotQA	NQ	HotpotQA	NQ	HotpotQA
<i>Top k=1</i>						
LLM (w/o RAG)	1096	1073	1188	1270	47	47
LLM (with RAG)	1092	1006	3186	2694	60	57
LLM (with RAG*)	1396	1024	3001	2570	69	63
LLMLingua-2	921 (-34.0%)	716 (-30.1%)	2431 (-19.0%)	2082 (-19.0%)	84 (-21.7%)	76 (-20.6%)
RECOMP	754 (-46.0%)	563 (-45.0%)	2101 (-30.0%)	1799 (-30.0%)	98 (-42.0%)	89 (-41.3%)
ICAE	841 (-39.8%)	738 (-28.0%)	3731 (-24.3%)	3293 (-28.1%)	169 (-145.0%)	164 (-160.3%)
xRAG	1295 (-7.2%)	1188 (-16.0%)	1317 (-56.1%)	1158 (-54.9%)	70 (-1.4%)	71 (-12.7%)
COCOM	488 (-65.0%)	541 (-47.2%)	2346 (-21.8%)	1905 (-25.9%)	138 (-100.0%)	138 (-119.0%)
PISCO	502 (-64.0%)	553 (-46.0%)	2251 (-25.0%)	1927 (-25.0%)	142 (-105.8%)	141 (-123.8%)
SeleCom	535 (-61.7%)	496 (-51.6%)	1166 (-61.1%)	1271 (-50.6%)	49 (-29.0%)	49 (-22.2%)
<i>Top k=5</i>						
LLM (with RAG*)	1537	1459	10811	4038	165	80
SeleCom	590 (-61.6%)	505 (-65.4%)	1664 (-84.6%)	1416 (-64.9%)	72 (-56.4%)	53 (-33.8%)

6 Extended Experiments

In this section, we evaluate how **SeleCom** addresses the limitations of full-compression methods (§6.1), **SeleCom**'s generalization and scalability (§6.2), and ablation on training strategies (§6.3). We also present ablations on projector and normalization (Appendix E),

sensitivity to p and n (Appendix F) and robustness under poor retrieval (Appendix G).

6.1 Extended Analysis

6.1.1 SeleCom in instruction following. Under the same setup as §3.2, we visualize attention of the generator in **SeleCom** during decoding. As shown in Figure 6, when doing reconstruction, attention peaks correctly occur at the instruction, query, and <DOCUMENT> area. When doing instruction-following, attention shifts almost entirely to the instruction, assigning negligible mass to <DOCUMENT> while still producing correct responses. This behavior contrasts with full-compression methods, demonstrating the intact instruction-following capability of **SeleCom**.

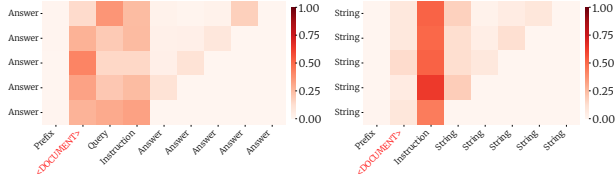


Figure 6: Attention heatmap of **SeleCom**'s generator under QA (left) and instruction-following (right) prompts.

6.1.2 SeleCom in information selection. Under the same setup as §3.3, we visualize the selector's attention while generating the <ENCODE> embeddings. As shown in Figure 7, the heatmap shows sharp, localized peaks on query and query-relevant document regions, while irrelevant regions are ignored. This pattern confirms that our selector acts as a query-conditioned information selector that yields cleaner, redundancy-free embeddings for the generator.

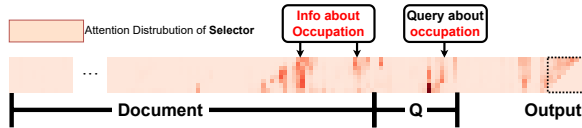


Figure 7: Attention of **SeleCom**'s selector under QA prompt.

6.2 Generalization and Scalability

We evaluate **SeleCom** across varying model scales and architectures (Table 3). Tested on Qwen2.5-3B and Llama-2-13B generators, **SeleCom** consistently demonstrates performance parallel to RAG baselines. This confirms that query-conditioned selection effectively preserves reasoning cues regardless of generator model family and capacity. Using different backbones of selectors also maintains competitive performance, despite expected performance drop due to reduction in the selector size. These results confirm that the **SeleCom** framework can effectively generalize and scale to different backbone sizes and architectures.

6.3 Ablation on Training Strategy

We ablate four training strategy: (I) w/o stage1 ft, which removes the first-stage selector training; (II) w/o stage2 ft, which skips the

Table 3: Robustness over generator and selector backbones, including sizes and architectures.

Method	Backbone	NQ			HotpotQA		
		EM	F1	LLM	EM	F1	LLM
<i>Generator (Fixed Selector: Qwen3-0.6B)</i>							
LLM w/ RAG*	Qwen2.5-3B	33.96	42.77	44.11	35.54	48.40	52.30
SeleCom		32.86	41.01	42.19	40.34	53.31	52.81
LLM w/ RAG*	Llama2-13B	46.00	54.63	56.08	42.85	57.11	57.16
SeleCom		44.99	53.59	54.99	46.21	59.59	59.78
<i>Selector (Fixed Generator: Mistral-7B)</i>							
SeleCom	GPT2-small	39.83	48.61	50.88	42.43	56.09	55.33
SeleCom	SmolLM-360m	42.85	51.65	53.38	47.54	60.95	61.03
SeleCom	Qwen3-Embedding-0.6B	41.80	49.72	50.91	44.46	57.88	57.49

second-stage generator fine-tuning; (III) stage2 (selector tuned), which jointly tunes the selector and generator in stage two; and (IV) Ours, which integrates both stages.

As shown in Figure 8, both stages contribute positively. Without stage1 training (w/o stage1 ft), performance on three metrics drops markedly, indicating that the selector requires explicit supervision to extract query-conditioned information. Removing stage2 fine-tuning (w/o stage2 ft) leads to weaker generator adaptation, reflecting the importance of teaching the generator how to utilize compressed embeddings. Stage2 (selector tuned) improves over either stage alone, but still lags behind our two-stage process.

We also validates the effectiveness of curriculum learning. Without curriculum learning, F1-Score drops by 0.2% to 0.5% across all datasets. Curriculum learning stabilizes optimization, allowing the model to progressively develop robust extraction capabilities.

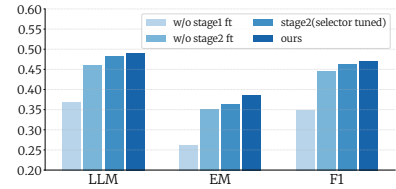


Figure 8: Ablation study on different training strategy.

7 Conclusion

This paper revisited the full-compression paradigm of soft compression RAG methods and demonstrates its infeasibility and non-necessity. We introduced **SeleCom** that shifts the role of encoder as a query-conditioned selector to preserve critical evidence under high compression. We carried out a comprehensive data synthesis framework to curate a massive and diverse dataset to train the selector. We validated that shifting to an autoregressive selection paradigm effectively prevents information dilution and aligns compressed representations with downstream generation behaviors. Experiments confirm that **SeleCom** achieves state-of-the-art performance while significantly reducing computation and latency. We open-source our code¹, data² and model checkpoints³.

¹<https://github.com/yhliu7458/SeleCom>

²https://huggingface.co/datasets/Ryan7458/SeleCom_Training

³<https://huggingface.co/Ryan7458/SeleCom>

References

- [1] Chen Amiraz, Florin Cuconasu, Simone Filice, and Zohar S. Karmin. 2025. The Distracting Effect: Understanding Irrelevant Passages in RAG. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2025.
- [2] Sotiris Anagnostidis, Dario Pavllo, Luca Biggio, et al. 2023. Dynamic Context Pruning for Efficient and Interpretable Autoregressive Transformers. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*.
- [3] Akari Asai, Zeqiu Wu, Yizhong Wang, et al. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.
- [4] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. 2013. Semantic Parsing on Freebase from Question-Answer Pairs. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, EMNLP 2013*.
- [5] Xin Cheng, Xun Wang, Xingxing Zhang, et al. 2024. xRAG: Extreme Context Compression for Retrieval-augmented Generation with One Token. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024*.
- [6] Darren Edge, Ha Trinh, Newman Cheng, et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *CoRR* abs/2404.16130 (2024). arXiv:2404.16130 doi:10.48550/ARXIV.2404.16130
- [7] Kavin Ethayarajh. 2019. How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019*.
- [8] Yunfan Gao, Xun Xiong, Xinyu Gao, et al. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *CoRR* abs/2312.10997 (2023). arXiv:2312.10997
- [9] Tao Ge, Jing Hu, Lei Wang, et al. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.
- [10] Yang Guo, Yutian Tao, Yifei Ming, et al. 2025. Retrieval-Augmented Generation as Noisy In-Context Learning: A Unified Theory and Risk Bounds. *CoRR* abs/2506.03100 (2025). arXiv:2506.03100
- [11] Kelvin Guu, Kenton Lee, Zora Tung, et al. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. *CoRR* abs/2002.08909 (2020). arXiv:2002.08909 <https://arxiv.org/abs/2002.08909>
- [12] Katharina Hammerl, Alina Fastowski, Jindrich Libovický, and Alexander Fraser. 2023. Exploring Anisotropy and Outliers in Multilingual Language Models for Cross-Lingual Semantic Sentence Similarity. In *Findings of the Association for Computational Linguistics: ACL 2023*.
- [13] Yizheng Huang and Jimmy Huang. 2024. A Survey on Retrieval-Augmented Text Generation for Large Language Models. *CoRR* abs/2404.10981 (2024). arXiv:2404.10981 doi:10.48550/ARXIV.2404.10981
- [14] Gautier Izacard and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, EACL 2021*.
- [15] Gautier Izacard, Patrick Lewis, Maria Lomeli, et al. 2023. Atlas: Few-shot Learning with Retrieval Augmented Language Models. *J. Mach. Learn. Res.* 24 (2023), 251:1–251:43.
- [16] Siddharth Jha, Lutfi Eren Erdogan, Sehoon Kim, et al. 2024. Characterizing Prompt Compression Methods for Long Context Inference. *CoRR* abs/2407.08892 (2024). arXiv:2407.08892 doi:10.48550/ARXIV.2407.08892
- [17] Robin Jia and Percy Liang. 2017. Adversarial Examples for Evaluating Reading Comprehension Systems. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, EMNLP 2017*.
- [18] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, et al. 2023. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023*.
- [19] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017*.
- [20] Jiho Kim, Sungjin Park, Yeonsu Kwon, et al. 2023. FactKG: Fact Verification via Reasoning on Knowledge Graphs. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2023.
- [21] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. 2019. Natural Questions: a Benchmark for Question Answering Research. *Trans. Assoc. Comput. Linguistics* 7 (2019), 452–466.
- [22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020*.
- [23] Jiarui Li, Ye Yuan, and Zehua Zhang. 2024. Enhancing LLM Factual Accuracy with RAG to Counter Hallucinations: A Case Study on Domain-Specific Queries in Private Knowledge-Bases. *CoRR* abs/2403.10446 (2024). arXiv:2403.10446
- [24] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing Context to Enhance Inference Efficiency of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023*.
- [25] Yongqi Li, Xinyu Lin, Wenjie Wang, et al. 2024. A Survey of Generative Search and Recommendation in the Era of Large Language Models. *CoRR* abs/2404.16924 (2024). arXiv:2404.16924 doi:10.48550/ARXIV.2404.16924
- [26] Zhuowan Li, Cheng Li, Mingyang Zhang, et al. 2024. Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: EMNLP 2024 - Industry Track*.
- [27] Zongqian Li, Yinhong Liu, Yixuan Su, and Nigel Collier. 2025. Prompt Compression for Large Language Models: A Survey. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers*.
- [28] Jintao Liang, Gang Su, Huifeng Lin, You Wu, Rui Zhao, and Ziyue Li. 2025. Reasoning RAG via System 1 or System 2: A Survey on Reasoning Agentic Retrieval-Augmented Generation for Industry Challenges. *CoRR* abs/2506.10408 (2025). arXiv:2506.10408 doi:10.48550/ARXIV.2506.10408
- [29] Nelson F. Liu, Kevin Lin, John Hewitt, et al. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Trans. Assoc. Comput. Linguistics* 12 (2024), 157–173.
- [30] Ilya Loshchilov, Cheng-Ping Hsieh, Simeng Sun, and Boris Ginsburg. 2025. nGPT: Normalized Transformer with Representation Learning on the Hypersphere. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*.
- [31] Maxime Louis, Hervé Déjean, and Stéphane Clinchant. 2025. PISCO: Pretty Simple Compression for Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics, ACL 2025*. <https://aclanthology.org/2025.findings-acl.800/>
- [32] Xinbei Ma, Yeyun Gong, Pengcheng He, et al. 2023. Query Rewriting for Retrieval-Augmented Large Language Models. *CoRR* abs/2305.14283 (2023). arXiv:2305.14283 doi:10.48550/ARXIV.2305.14283
- [33] Alex Mallen, Akari Asai, Victor Zhong, et al. 2023. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2023.
- [34] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, et al. 2024. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of the Association for Computational Linguistics, ACL 2024*.
- [35] David Rau, Shuai Wang, Hervé Déjean, and Stéphane Clinchant. 2024. Context Embeddings for Efficient Answer Generation in RAG. *CoRR* abs/2407.09252 (2024). arXiv:2407.09252 doi:10.48550/ARXIV.2407.09252
- [36] Claude E. Shannon. 1948. A mathematical theory of communication. *Bell Syst. Tech. J.* 27, 4 (1948), 623–656.
- [37] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2020. Efficient Transformers: A Survey. *CoRR* abs/2009.06732 (2020). arXiv:2009.06732
- [38] William Timkey and Marten van Schijndel. 2021. All Bark and No Bite: Rogue Dimensions in Transformer Language Models Obscure Representational Quality. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021*.
- [39] S. M. Towhidul Islam Tonmoy, S. M. Mehedi Zaman, Vinija Jain, et al. 2024. A Comprehensive Survey of Hallucination Mitigation Techniques in Large Language Models. *CoRR* abs/2401.01313 (2024). arXiv:2401.01313 doi:10.48550/ARXIV.2401.01313
- [40] Sourav Verma. 2024. Contextual Compression in Retrieval-Augmented Generation for Large Language Models: A Survey. *CoRR* abs/2409.13385 (2024). arXiv:2409.13385 doi:10.48550/ARXIV.2409.13385
- [41] Tu Vu, Mohit Iyyer, Xuezhi Wang, et al. 2024. FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation. In *Findings of the Association for Computational Linguistics, ACL 2024*.
- [42] Shitao Xiao, Zheng Liu, Yingxia Shao, and Zhao Cao. 2022. RetroMAE: Pre-Training Retrieval-oriented Language Models Via Masked Auto-Encoder. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022*.
- [43] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.
- [44] Zhilin Yang, Peng Qi, Saizheng Zhang, et al. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2369–2380.
- [45] Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2024. Making Retrieval-Augmented Language Models Robust to Irrelevant Context. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net.

- [46] Hao Yu, Aoran Gan, Kai Zhang, et al. 2024. Evaluation of Retrieval-Augmented Generation: A Survey. *CoRR* abs/2405.07437 (2024). arXiv:2405.07437 doi:10.48550/ARXIV.2405.07437
- [47] Wenhao Yu, Dan Iter, Shuohang Wang, et al. 2023. Generate rather than Retrieve: Large Language Models are Strong Context Generators. In *The Eleventh International Conference on Learning Representations, ICLR 2023*. OpenReview.net.
- [48] Yun Zhu, Jia-Chen Gu, Caitlin Sikora, et al. 2025. Accelerating Inference of Retrieval-Augmented Generation via Sparse Context Selection. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*. OpenReview.net.

A Analysis Details

A.1 Infeasibility of Full Compression

A.1.1 Detailed theory. (1) The full-compression task first pushes the compressed embeddings Z towards a "positive half-space" of the LLM's token embeddings, causing direction similarity that results in positive attention scores.

Under a hyperspherical view, each query vector in decoder's attention layer can be decomposed into a unit direction (on-sphere) and a radius (norm). Modern decoder-only LLM representations are markedly anisotropic—mass concentrates in a narrow cone and a few high-variance "outlier" features dominate the similarity structure [7, 12, 30, 38]. This implies the existence of a "positive half-space" direction v such that the vast majority of tokens $E = \{e_w\}_{w \in V} \subset \mathbb{R}^d$ lie in its positive half-space: $\Pr_{w \sim V} [\cos(e_w, v) > 0] \approx 1$.

During the full-compression training task, the model's desired output fully rely on the Z , which forces the attention score on Z to consistently increase. This forces Z to gradually align toward this direction v , "hacking" the generator attention layer to produce positive attention scores on these embeddings.

This sets up the next cause: (2) Once the embedding direction is aligned, the everlasting training pressure generated by the full-compression task will push the norm R_Z of the Z higher and higher, systematically amplifying attention logits that causes polarized attention toward Z .

Specifically, the compressed embeddings Z must contain as much information as possible from the retrieved document for the downstream LLM to correctly reconstruct the document. From the information theory perspective, we refer to the Shannon–Hartley theorem [36]: $C = B \log_2(1 + \frac{S}{N})$, where C is the expressing capacity of the Z , B is the channel bandwidth determined by the LLM, and N is the noise power. With the decoder frozen and noise fixed, Z 's expressing capacity monotonically increases with S , the signal power, which corresponds to the the embedding norm R_Z . Thus, training effectively inflates the embedding's magnitude to transmit more information reliably across layers and tokens.

Furthermore, the strong pressure to raise attention scores also inherently pushes up R_Z , which serves as the *key vector* of all decoded tokens in the attention mechanism.

(3) While the full-compression task does not enforce instruction-following abilities, attention scores to other tokens remain low. Consequently, once the direction of the Z is correct, and R_Z grows high, softmax normalization rapidly polarizes attention toward Z . This drives attention to collapse onto the compressed document embeddings across layers. When prompts include special instructions or auxiliary information, the LLM simply ignores them and keeps their pre-defined behaviors as if it sees only the compressed embeddings. Consequently, the model's ability to analyze the query

and extract supporting evidence deteriorates, which further reduces its performance on downstream tasks.

A.1.2 Detailed experimental setup. A pipeline of *Qwen3-Embedding-0.6B* as the full-compression encoder, a frozen *Qwen2.5-7B-Instruct* as the generator and a one-layer MLP as the projector is trained on the document reconstruction task with 10,000,000 random samples from the wikipedia dump. We then apply different prompts to test its query-conditioned generation ability.

A.2 Non-necessity of Full Compression

A.2.1 Detailed theory. Let Q be the query, D as the retrieved document, A as the answer, and $E \subseteq D$ the necessary information, with $N = D \setminus E$ as the set of noisy information. According to the definition of E , we have:

$$I(A; D | Q) \approx I(A; E | Q) + I(A; N | Q) = I(A; E | Q).$$

Let $Z = f(D)$ be the full-compression encoder's document embeddings, and $SZ_E = f(E)$ and $SZ_N = f(N)$ respectively be the necessary and noisy information compressed. Z is a capacity-limited and query-agnostic function of (E, N) , part of its bitrate is inevitably spent preserving information about N , yielding:

$$I(A; Z | Q) < I(A; Z_E | Q) + I(A; Z_N | Q) = I(A; Z_E | Q).$$

Thus, full compression with no focus retains strictly less helpful information than compressing only the query-conditioned necessary information.

On the other hand, if we directly build a selector that given Q , it extracts and compresses only the necessary information as $Z_s = f_s(D) = f_s(E)$. Its superior capability than the full-compression encoder can be strictly proven:

$$I(A; Z_s | Q) = I(A; Z_E | Q) > I(A; Z | Q).$$

In this case, the selector's document embeddings carries information that *aligns with the query* and is only *slightly less than* the baseline, while full compression is strictly worse due to capacity spent on N and lossy entanglement of E .

A.2.2 Detailed experimental setup. The baseline RAG results are obtained with *Qwen2.5-7B-Instruct*. The full-compression results are obtained with the trained pipeline during the infeasibility analysis. Under the same document, the baseline RAG is query-conditioned and the full-compression encoder is not.

B Main Experiment settings

All experiments are completed on a Ubuntu 22.04.1 LTS server with a Intel(R) Xeon(R) Gold 6530 CPU and 8×NVIDIA GeForce RTX 4090 GPU (48GB) installed. We conduct stage1 training by 1 epoch, with batch size = 10 per device and learning rate = 0.00005. We perform stage2 training by 3 epoch, using LoRA $r = 64$, batch size = 3 per device and learning rate = 0.0001. For training time, Stage1 takes ~ 40h, Stage2 takes ~ 12h.

C Notations

Definitions of the notations are provided in Table 4

D Comprehensive Experimental results

We present the efficiency results on all datasets in Table 8.

Table 4: Notations used in section Methodology.

Symbol	Description
q_i, d_i, a_i	The i -th query, document, and answer triple.
k	Number of retrieved documents.
P_{select}, P_{gen}	Prompts for the Selector and Generator.
T_i	Token sequence for the i -th input.
p	Number of special $\langle En \rangle$ tokens for selection.
n	Number of embeddings per document.
$H_{i:p}^i$	Hidden vectors corresponding to $\langle En \rangle$ tokens.
E_i	Compressed embeddings set for document d_i .
$\langle emb \rangle$	Special placeholder token.
S_i	Embedded sequence of P_{gen} before inserting E_i .
$\hat{S}_i$	Final embedded sequence after inserting E_i .
$\text{Tok}(\cdot)$	Tokenization function.
$\text{Emb}(\cdot)$	Token embedding function.
$\text{Insert}(\cdot)$	Function inserting E_i into placeholder positions.
$\text{Select}(\cdot)$	The Selector model.
$\text{Proj}(\cdot)$	The Projector model.
$\text{Split}_n(\cdot)$	Function splitting a vector into n sub-vectors.
$\mathcal{L}_{CE}$	Cross-entropy loss function.

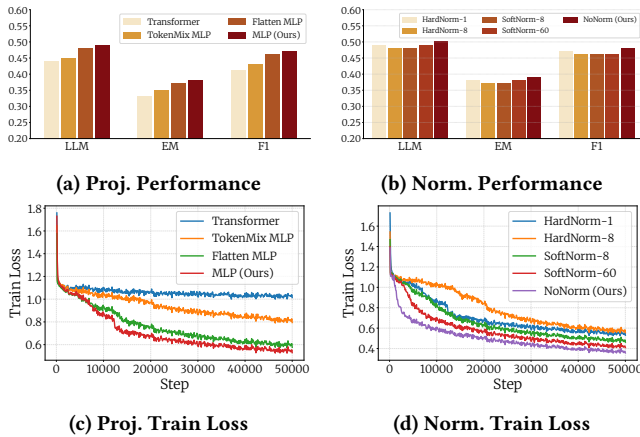
E Ablation on Projector and Normalization

We evaluate four projector designs: (1) Transformer-based projector; (2) Token-wise MLP followed by learnable aggregation; (3) Flatten two-layer MLP with LayerNorms; (4) our one-layer MLP.

For normalization, we evaluate HardNorm- n , which enforces strict normalization to the norm value of n ; SoftNorm- n , which rescales embeddings by a smooth norm-target function; and our NoNorm, which imposes no explicit constraint.

We also evaluate choices for compressed embedding normalization: HardNorm- n enforcing strict normalization to the norm value of n ; SoftNorm- n applying smooth rescaling target; our NoNorm, with no normalization at all.

Results are shown in Figure 9. For projectors, our lightweight MLP achieves the best performance and fastest convergence. For normalization, both HardNorm and SoftNorm lead to suboptimal performance. These findings strengthen our design choices.

**Figure 9: Ablation on normalization and projector design.**

F Sensitivity to p and n

We examine the impact of number of selector tokens (p) and compressed embeddings (n). As shown in Table 5, **SeleCom** remains robust across configurations.

Table 5: Sensitivity on p and n .

Config.		NQ			HotpotQA		
p	n	EM	F1	LLM	EM	F1	LLM
8	1	41.86	50.07	51.33	45.66	58.05	57.30
8	4	41.78	49.61	51.15	44.90	57.54	56.86
4	2	41.80	50.09	51.97	45.43	58.04	58.60
16	2	41.94	50.04	50.89	44.10	57.34	57.45
8	2	41.80	49.72	50.91	44.46	57.88	57.49

G Robustness to Irrelevant Context

We evaluate robustness against retrieval failure by replacing top- k documents with random corpus samples. As shown in Table 6, we report the F1 degradation ($\Delta F1$) compared to standard retrieval. **SeleCom** demonstrates robustness parallel to the strongest non-compression baseline ($\max(\text{RAG}, \text{RAG}^*)$). Notably, on multi-hop QA (HotpotQA), SeleCom exhibits smaller performance drops, confirming that the query-conditioned selector effectively filters out noise that distracts the generator.

Table 6: Performance drop ($\Delta F1 \downarrow$) under random documents.

Method	NQ	TriviaQA	WebQA	PopQA	HotpotQA	FactKG
Baseline	3.6%	7.1%	1.1%	13.8%	15.7%	10.7%
SeleCom	3.7%	6.0%	1.0%	14.0%	11.1%	4.1%

H Human Verification on LLM Usage

For dataset synthesis, we randomly select 10 human experts to verify the quality of 1000 samples from the constructed dataset. They report quality of document, factuality and supportability of QA, and human alignment for difficulty labeling. For LLM-as-a-judge, we randomly select 6 human experts, each verifying 200 judgement samples for each evaluation dataset of **SeleCom**. They report judgement consistency. Results are shown in Table 7, which validates the quality of the LLM-synthetic dataset and the utility of LLM-as-a-judge.

Table 7: Human verification results on LLM usage

Component	Criterion	Result (%)
Synthetic Data	Document Quality	98.2
	Factuality	99.5
	Supportability	99.9
	Difficulty Label	91.0
LLM-as-a-judge	Score Consistency	97.5

Table 8: Complete efficiency results on six datasets with three evaluation metrics.

Models	Natural Questions			TriviaQA			Web Questions			PopQA			HotpotQA			FactKG		
	TIL	GFLOPs	TI/FT	TIL	GFLOPs	TI/FT	TIL	GFLOPs	TI/FT	TIL	GFLOPs	TI/FT	TIL	GFLOPs	TI/FT	TIL	GFLOPs	TI/FT
Mistral-7B-Instruct																		
<i>Top k=1</i>																		
LLM (w/o RAG)	1096	1188	47	941	1218	55	990	1144	47	953	1080	47	1072	1270	47	1087	1286	48
LLM (with RAG)	1092	3186	60	956	3216	61	998	3043	59	810	2778	57	1006	2694	57	1162	3037	60
LLM (with RAG*)	1396	3001	69	956	3101	73	1430	2879	69	881	3688	66	1024	2570	63	743	3002	68
LLMLingua-2	921 (-34.0%)	2431 (-19.0%)	84 (-21.7%)	655 (-30.0%)	2512 (-19.0%)	88 (-20.5%)	986 (-31.0%)	2332 (-19.0%)	82 (-18.8%)	617 (-30.0%)	2177 (-19.0%)	79 (-19.7%)	716 (-30.1%)	2082 (-19.0%)	76 (-20.6%)	520 (-30.0%)	2432 (-19.0%)	81 (-19.1%)
RECAMP	754 (-46.0%)	2101 (-30.0%)	98 (-42.0%)	515 (-45.0%)	2171 (-30.0%)	103 (-41.1%)	786 (-45.0%)	2015 (-30.0%)	97 (-40.6%)	485 (-45.0%)	1882 (-30.0%)	92 (-39.4%)	563 (-45.0%)	1799 (-30.0%)	89 (-41.3%)	409 (-45.0%)	2101 (-30.0%)	95 (-39.7%)
ICAE	841 (-39.8%)	3731 (-24.3%)	169 (-145.0%)	615 (-34.3%)	3812 (-22.9%)	165 (-136.0%)	929 (-35.0%)	3683 (-27.9%)	171 (-147.8%)	699 (-20.7%)	3186 (-18.5%)	160 (-142.4%)	738 (-28.0%)	3293 (-28.1%)	164 (-160.3%)	531 (-28.3%)	3709 (-28.6%)	154 (-126.5%)
xRAG	1295 (-7.2%)	1317 (-56.1%)	70 (-1.4%)	1309 (-39.9%)	1190 (-61.6%)	38 (-47.9%)	1349 (-5.7%)	1336 (-53.6%)	74 (-7.2%)	1208 (-37.1%)	942 (-65.0%)	78 (-18.2%)	1188 (-16.0%)	1158 (-54.9%)	70 (-11.1%)	844 (-13.6%)	1336 (-55.5%)	68 (0.0%)
COCO	488 (-65.0%)	2346 (-21.8%)	138 (-100.0%)	368 (-60.7%)	2459 (-20.7%)	134 (-83.6%)	642 (-55.1%)	2511 (-19.7%)	137 (-98.6%)	514 (-41.7%)	1841 (-31.5%)	135 (-104.5%)	540 (-47.3%)	1905 (-25.9%)	137 (-117.5%)	478 (-35.7%)	2335 (-22.2%)	144 (-111.8%)
PISCO	302 (-64.0%)	2251 (-25.0%)	142 (-105.8%)	384 (-59.0%)	2326 (-25.0%)	139 (-90.4%)	658 (-54.0%)	2159 (-25.0%)	140 (-102.9%)	529 (-40.0%)	2016 (-25.0%)	136 (-106.1%)	553 (-46.0%)	1927 (-25.0%)	141 (-123.8%)	490 (-34.1%)	2352 (-25.0%)	145 (-113.2%)
SeleCom	535 (-61.7%)	1166 (-61.1%)	49 (-29.0%)	528 (-43.6%)	1270 (-59.0%)	50 (-31.5%)	565 (-60.5%)	1144 (-60.3%)	50 (-27.5%)	504 (-42.8%)	1120 (-58.3%)	48 (-27.3%)	496 (-51.6%)	1271 (-50.5%)	49 (-22.2%)	434 (-41.6%)	1384 (-53.9%)	51 (-25.0%)
<i>Top k=3</i>																		
LLM (with RAG*)	1537	10811	165	1498	11196	171	1514	10385	159	1458	9221	142	1459	4038	80	1533	10674	163
SeleCom	590 (-61.6%)	1664 (-84.6%)	72 (-56.4%)	534 (-64.3%)	1764 (-84.2%)	75 (-56.1%)	1207 (-20.3%)	1787 (-82.8%)	73 (-54.1%)	676 (-53.6%)	1651 (-82.1%)	76 (-46.5%)	505 (-65.4%)	1416 (-64.9%)	53 (-33.8%)	1050 (-31.5%)	2034 (-80.9%)	78 (-52.1%)
Qwen2.5-7B-Instruct																		
<i>Top k=1</i>																		
LLM (w/o RAG)	792	983	36	386	918	35	851	990	35	551	852	37	449	950	36	627	933	36
LLM (with RAG)	826	2749	44	368	2782	40	667	2785	38	661	2144	67	499	2273	40	524	2689	48
LLM (with RAG*)	645	2576	64	426	2703	54	452	2526	51	622	2192	64	670	2220	65	491	2694	62
SeleCom	433 (-32.9%)	961 (-67.7%)	48 (-25.0%)	423 (-0.7%)	1047 (-61.3%)	48 (-11.1%)	404 (-45.5%)	947 (-65.5%)	47 (-7.8%)	490 (-21.5%)	962 (-56.1%)	58 (-54.5%)	513 (-23.4%)	1063 (-52.1%)	53 (-18.5%)	401 (-18.3%)	1521 (-45.5%)	50 (-19.4%)
<i>Top k=5</i>																		
LLM (with RAG*)	733	9682	142	731	10004	147	734	9450	141	708	7989	121	691	3564	73	723	9522	151
SeleCom	518 (-29.3%)	1454 (-85.0%)	65 (-54.2%)	519 (-29.0%)	1541 (-84.6%)	66 (-55.1%)	545 (-25.7%)	1401 (-85.2%)	69 (-51.1%)	477 (-32.6%)	1478 (-81.5%)	73 (-39.7%)	515 (-25.5%)	1228 (-65.5%)	58 (-20.5%)	589 (-18.5%)	1663 (-82.5%)	71 (-53.0%)

I Case Study

Examples of full compression not following instructions:

Input Prompt:

Background: DOCUMENT
Who is Republic of Hawaii’s co-founder?

Generator Output under full-compression:

Title: O. M. Hustvedt
Content: ...

Generator Output without compression:

The Republic of Hawaii’s co-founder mentioned in the background information is Henry E. Cooper.

Input Prompt:

Background: DOCUMENT
Why does Olaf M. Hustvedt’s son dropped from the academy?

Generator Output under full-compression:

Title: Olaf M. Hustvedt
Content: ...

Generator Output without compression:

According to the information provided, Olaf M. Hustvedt’s son, Erling H. Hustvedt, was dropped from the U.S. Naval Academy due to an incorrect evaluation of his eyesight.

Cases of redundant/misleading retrieved documents:

Document:

Title: God of War (2018 video game)
Content: God of War is ... It was released worldwide on April 20, 2018, for the PlayStation 4 (PS4) with a Microsoft Windows version set to release on January 14, 2022. The game is the eighth installment in the God of War series, the eighth chronologically, and the sequel to 2010’s God of War III.

Input Prompt: when was the last god of war made?

Baseline RAG Output: 2010

SeleCom Output: 2018

Document:

Title: The A-Team
Content: Although the part of Face was written by Frank Lupo and Stephen J. Cannell with Dirk Benedict in mind, NBC insisted that the part should be played by another actor. Therefore, in the pilot, Face was portrayed by Tim Dunigan, who was later replaced by Benedict, with the comment that Dunigan was “too tall and too young”.

Input Prompt: who played the face in the a team?

Baseline RAG Output: Tim Dunigan

SeleCom Output: Dirk Benedict

J Prompts

P_{select}:

Document
[Document]
Question
[Question]
Instruction
Extract the key information from the document that is helpful to answer the question.
-ENCODE>...<ENCODE-

P_{gen}:

Reference
<DOCUMENT_START><DOCUMENT>...<DOCUMENT><DOCUMENT_END>
Question
[Question]
Instruction
Answer the question according to the reference provided above.
Restriction
1. You must use English.
2. You must DIRECTLY provide the answer in this STRICT format: <answer></answer>.
3. You must not generate any other text.

Prompt for document content filtering:

Instruction
You will be given a document. Verify if the document is a piece of human-readable plain text.
Document
[DOCUMENT]
Criteria
ALL PART of the document must look like a piece of descriptive text. If it seems to be a piece of code, table, figure or other non-descriptive elements, it is considered of BAD quality. You must be a strict judge and consider a text as BAD at any opportunity.
Restriction
1. You must generate your judgement in this STRICT format: <document></document>
2. You must not generate any other text.

Prompt for document quality filtering:

Instruction
You will be given a document. Evaluate the amount of information contained by the document.
Document
[DOCUMENT]
Criteria
You will score the amount of information of the document as an integer in 1,2,3,4,5,6,7,8,9,10.
Format restriction
1. You must generate your evaluation in this STRICT format: <response></response>
2. You must not generate any other text.

Prompts for EASY QA generation:

Document
[DOCUMENT]
Instruction
Ask a question about one single fact mentioned in the above document, and provide its answer. The answer should be short, concise and factual (1-10 words typically) and correctly answers the question.
Restriction
1. You must use English.
2. You must generate the question and its answer in this STRICT format: <question></question><answer></answer>.
3. You must not generate any other text.

Prompts for HARD QA generation:

Document
[DOCUMENT]
Instruction
Ask a challenging, real-world question about one single fact that can be inferred from the above document and provide its answer. The question should require reasoning, inference, or common sense beyond just looking up facts directly from the text. The question should never ask for more than one fact. The answer should be short, concise and factual (1-10 words typically) and correctly answers the question. Avoid questions that can be answered by simply copying text from the document.
Restriction
1. You must use English.
2. You must generate the question and its answer in this STRICT format: <question></question><answer></answer>.
3. You must not generate any other text.

Prompt for QA filtering:

Instruction
You will be given a document, a question on this document, and an answer to this question. Judge the correctness of the answer, then evaluate the difficulty of the question to be correctly answered given the document content.
Document
[DOCUMENT]
Question
[Question]
Answer
[Answer]
Criteria
- Judge the answer by if it is correct to answer the question.
If correct, CORRECT. If wrong, WRONG.
- Then score the difficulty of the question to be correctly answered given the document as an integer in 1,2,3,4,5.
Format restriction
1. You must generate your response in this STRICT format: <correctness></correctness><difficulty></difficulty>.
2. You must not generate any other text.

Prompt of LLM-as-a-judge metric:

You will be given a question, a candidate answer, and a reference answer. Judge whether the candidate answer correctly addresses the question compared to the reference. Respond “ONLY” with a numeric score between 0 (completely wrong) and 1 (perfectly correct).
Question
[Question]
Candidate Answer
[Prediction]
Reference Answer
[Ground truth]